Small Object Detection Using YOLO: Traffic Sign Detection

CSCI 611 – Computer Vision | Spring 2026 | Assignment 3

Amol Bhalerao | California State University, Chico

1. Introduction

Detecting traffic signs from vehicle-mounted cameras is one of the more practically relevant applications of object detection. It also happens to be challenging, signs are often small relative to the full frame, partially occluded, or captured at oblique angles. This report documents my work on training a YOLOv8 model specifically for this task, starting from a COCO-pretrained baseline and iteratively improving it through fine-tuning and configuration experiments.

The primary goal was to compare detection performance between (a) the off-the-shelf pretrained model, and (b) fine-tuned versions trained on a traffic sign dataset, with different image resolutions and augmentation settings.

2. Dataset Description and Preprocessing

2.1 Dataset Source

I used the **Road Signs Detection** dataset from Roboflow Universe — an existing, publicly available dataset found online. It can be accessed at:
https://universe.roboflow.com/roboflow-100/road-signs-6ih4y (version 1).

The dataset contains 2,093 images across 21 sign classes, pre-annotated with bounding boxes in YOLO format. The classes include a range of Indonesian traffic signs: parking area indicators, bus stop markers, pedestrian crossing signs, U-turn location signs, and lane direction signs, among others. Images were already resized to 640×640 pixels by Roboflow. Since this is a full publicly available dataset (not a custom subset), it was not uploaded to GitHub — only the link is provided per the assignment instructions.

Split	Images	Percentage
Train	1,376	65.7%
Validation	488	23.3%
Test	229	11.0%
Total	**2,093**	100%

Table 1: Dataset split sizes.

2.2 Preprocessing and Annotation

Since the dataset was already annotated in YOLO format (each image paired with a `.txt` file containing `class_id x_center y_center width height` per line, all normalized to [0,1]), no additional annotation

work was needed. I did verify the annotation quality by plotting ground-truth bounding boxes over a random sample of training images — the boxes were generally tight and accurate.

A bounding box size analysis showed that a significant portion of the annotations have a relative area below 1% of the image — confirming that this is indeed a small object detection problem where standard detection settings may not be sufficient.

2.3 Baseline Test Split

The 229 test images (≈11% of the full dataset) were held out and used consistently across all experiments to ensure fair comparison between the pretrained and fine-tuned models.

3. Experimental Setup

3.1 Model Architecture

I used **YOLOv8 nano (YOLOv8n)** from the Ultralytics library throughout all experiments. The nano variant was chosen because it is the fastest to train and run, making experimentation more practical. While larger variants (YOLOv8s, m, l) would likely produce better absolute results, the nano model is sufficient to demonstrate the methodology and compare configurations.

3.2 Anchor Box Tuning

YOLOv8 uses an **anchor-free detection head**, which means traditional anchor box tuning (as used in YOLOv5 and earlier) is not applicable here. Instead of predicting offsets from fixed anchor boxes, YOLOv8 directly predicts bounding box coordinates using Distribution Focal Loss (DFL). This actually benefits small object detection because the model is not constrained by pre-defined anchor scales — it can predict boxes of any size without needing to tune anchors for small objects specifically.

3.3 Training Configurations

Two fine-tuning configurations were tested on top of the COCO-pretrained weights:

Setting	Baseline (Pretrained)	Config A (640px)	Config B (1280px)
Image size	640	640	1280
Epochs	—	30	30
Batch size	—	8	4
Mosaic aug.	—	1.0	1.0
Scale jitter	—	0.5	0.7
Flip (lr/ud)	—	0.5 / 0.1	0.5 / 0.2

Random erasing	—	—	0.1
Rotation	—	±10°	±15°
Approx. train time	—	~17 min	~51 min

Table 2: Training configuration summary. All training done on Google Colab T4 GPU.

Config B uses a higher resolution (1280px) and more aggressive augmentation specifically intended to help with small object detection. The larger scale jitter (0.7) forces the model to recognize signs at many different scales, and random erasing simulates partial occlusion. The trade-off is a 3× longer training time and lower batch size due to GPU memory constraints.

4. Results and Analysis

4.1 Overall Comparison

Model	mAP@50	mAP@50-95	Precision	Recall
Pre-trained YOLOv8n (COCO)	0.000	0.000	0.000	0.000
Fine-tuned Config A (640px)	**0.954**	**0.817**	0.934	**0.908**
Fine-tuned Config B (1280px)	0.947	0.752	0.934	0.876

Table 3: Test set metrics for all three model configurations.

The pretrained COCO model scored zero across all metrics on the test set. This was expected — the COCO dataset does not include traffic sign categories matching this dataset's classes, so the model has no basis for detecting them. It essentially detected nothing relevant, which underlines how important domain-specific fine-tuning is even for a model as capable as YOLOv8.

After just 30 epochs of fine-tuning, Config A reached a mAP@50 of **0.954** — a dramatic improvement. The model learned to recognize all 21 sign classes from scratch within a short training run. By epoch 1, mAP@50 was already at 0.172, and it climbed steadily through all 30 epochs without showing obvious overfitting (validation loss continued to decrease in parallel with training loss).

Config B (1280px) achieved a slightly lower mAP@50 (0.947) and notably lower mAP@50-95 (0.752 vs. 0.817). This was somewhat surprising — I expected the higher resolution to help with small objects. In hindsight, the reduced batch size (4 vs. 8) likely made training less stable, and 30 epochs may not have been enough for the model to fully converge at 1280px. The higher resolution also means the feature pyramid covers a different scale range, which may need longer training to adapt to. Config B did however take 3× longer to train.

4.2 Confidence Threshold Sweep

I swept the confidence threshold from 0.1 to 0.7 on the Config B model. The results showed a typical precision-recall tradeoff: lower confidence thresholds increase recall but bring in more false positives, while higher thresholds improve precision but miss detections. The best F1 score was found around a threshold of **0.3–0.35**. Using the default threshold of 0.25 was reasonably close to optimal.

One observation worth noting: for small objects, erring on the side of lower confidence thresholds is generally preferred. Missing a traffic sign (false negative) is more dangerous in a real autonomous driving context than a false positive. This guided the decision to keep the inference threshold at 0.25 for the final evaluation.

4.3 NMS / IoU Threshold Sweep

The NMS IoU threshold controls when two overlapping predictions are considered duplicates and one is suppressed. For this dataset, an IoU threshold of **0.5–0.6** produced the best mAP@50. Using a very low IoU threshold (e.g., 0.3) caused too much suppression of nearby signs, hurting recall. Using a high threshold (0.8) let in duplicate predictions, reducing precision.

4.4 False Positive and False Negative Analysis

Running inference on the 229 test images with the Config B model (conf > 0.5), a total of 216 images had at least one high-confidence detection. Visual inspection of a random sample of results revealed two recurring patterns:

- **False Negatives:** The most common misses were on signs that appeared at the very edge of the frame, in heavy shadow, or at a steep angle. These are exactly the cases where small pixel coverage combined with low contrast made it hard for the model to fire with sufficient confidence. Lowering the confidence threshold from 0.5 to 0.25 recovered most of these missed detections at the cost of a small increase in false positives.
- **False Positives:** A small number of false positives appeared on background textures that visually resembled sign shapes — for example, rectangular wall markings or colored road surface markings. These tended to have low confidence scores (0.25–0.35) and were largely eliminated by raising the threshold to 0.4 or above.

This analysis confirmed that a confidence threshold of 0.3 offers the best balance for this dataset — capturing most true detections while suppressing the majority of false positives. For safety-critical deployment (such as a driver assistance system), a threshold of 0.25 would be preferable to minimize missed detections.

5. Challenges in Detecting Small Objects

Working through this assignment highlighted several challenges specific to small object detection:

1. **Low pixel coverage:** Many signs in the dataset occupy fewer than 32×32 pixels in the original frame. The backbone's downsampling reduces spatial resolution as features pass through deeper layers, making it progressively harder to retain the fine-grained detail needed for small object recognition.
2. **Class imbalance:** Some sign categories appeared hundreds of times in the training set, while a few were much rarer. This caused the model to perform better on frequent classes and struggle on underrepresented ones, as visible in the per-class rows of the confusion matrix.

3. **Appearance variability:** Even within a single sign class, lighting conditions, mounting angles, and distances created significant variation. Signs photographed from 30 meters away look very different from close-up shots, and the model needed enough augmentation diversity to handle both.
4. **Dataset domain:** The signs in this dataset are Indonesian traffic signs, which differ visually from American or European signs. A model trained on this dataset would need to be retrained or adapted before deployment in a U.S. context like Chico street footage.

6. Suggestions for Improvement

Based on the experiments, several directions could improve detection performance further:

- **Larger model variants:** YOLOv8n was used here for speed. Using YOLOv8s or YOLOv8m would increase model capacity and likely push mAP@50-95 higher without changing the training pipeline at all.
- **More epochs for Config B:** The 1280px model was likely undertrained at 30 epochs. Running it for 50–75 epochs would likely close the performance gap with Config A and potentially surpass it, especially on mAP@50-95 where bounding box localization precision matters more.
- **SAHI (Sliced Inference):** At inference time, slicing large images into overlapping tiles before running detection, then merging results, is a well-known technique for improving small object detection without any retraining. This would be a practical improvement for deployment on high-resolution dashcam footage.
- **Address class imbalance:** Oversampling rare classes during training or using weighted loss functions could help the model perform more uniformly across all 21 sign categories.
- **Additional data for Chico streets:** The professor provided Chico street footage that could be annotated and added to the training set. Even a small number of locally-annotated images would help the model adapt to the specific camera angle, lighting, and sign styles found in this region.

7. Conclusion

This assignment demonstrated that YOLOv8, when fine-tuned on domain-specific data, can achieve strong performance on traffic sign detection — going from 0% to 95.4% mAP@50 in 30 epochs of training. The experiments also showed that simply increasing image resolution does not guarantee better results; training stability, batch size, and the number of epochs all interact in ways that can offset the theoretical benefits of higher resolution.

The confidence threshold and NMS IoU sweep confirmed that the default Ultralytics settings (conf=0.25, iou=0.6) are close to optimal for this dataset, though a slightly lower confidence threshold (0.3) gave the best F1 score in practice.

The most important takeaway is that pretrained COCO weights, while powerful for general object detection, are essentially useless for specialized domains like traffic sign detection without fine-tuning. Domain adaptation through training on task-specific data is not optional — it is the baseline requirement.

References

1. Jocher, G., et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
2. Roboflow Road Signs Detection Dataset. https://universe.roboflow.com/roboflow-100/road-signs-6ih4y
3. Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767
4. Akyon, F. C., et al. (2022). Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. ICIP 2022.